

Domaine	Tâches	Outil	Rôle expliqué simplement
Dev	- Analyse des besoins - Conception - Développement - Tests - Revue de code	Git	Permet de sauvegarder le code et garder l'historique des modifications.
		GitHub / GitLab	Permet de partager le code avec l'équipe et travailler ensemble.
		VS Code / IntelliJ	Logiciels pour écrire et corriger le code.
		Maven / npm	Téléchargent et gèrent les bibliothèques nécessaires au projet.
		JUnit	Sert à tester automatiquement le code en Java pour vérifier qu'il fonctionne.
		Jest	Sert à tester automatiquement le code en JavaScript.
		SonarQube	Analyse le code pour détecter les erreurs, les failles de sécurité et les problèmes de qualité.
Ops	- Préparation infrastructure - Déploiement - Surveillance - Sécurité - Maintenance	Docker	Met l'application dans une "boîte" (conteneur) pour qu'elle fonctionne partout de la même manière.
		Kubernetes Gère plusieurs conteneurs Docker automatiquement (démarrage, arrêt, redémarrage). Orchestration = organisation et gestion automatique des conteneurs. • React → C'est le site web que tu vois (les boutons, images, textes... tout ce que l'utilisateur utilise). • Jest → Vérifie que tout fonctionne bien dans le site. ○ Exemple : le bouton "Envoyer" fonctionne correctement. • SonarQube → Vérifie que le code est propre et sans erreur. ○ Exemple : pas de faute dans le code, pas de bug caché. • Docker → Met ton site dans une boîte magique. ○ Cette boîte contient tout ce dont ton site a besoin pour marcher. ○ Tu peux donner cette boîte à quelqu'un d'autre, et ça marchera pareil sur son ordinateur. • Kubernetes → C'est le chef qui organise toutes les boîtes. ○ Il fait démarrer les boîtes quand il y a des visiteurs. ○ Il refait une boîte si elle tombe en panne. ○ Il crée plus de boîtes si beaucoup de gens visitent le site.	

		Jenkins Automatise les étapes : tester, vérifier, puis déployer l'application. Jenkins, GitLab CI/CD et GitHub Actions font presque la même chose. Leur rôle simple : Ce sont des outils de CI/CD. • CI (Intégration Continue) : À chaque fois qu'on envoie du code, l'outil teste automatiquement le projet (avec Jest, JUnit, etc.) et vérifie la qualité (avec SonarQube par exemple). • CD (Déploiement Continu) : Si tout est correct, l'outil peut automatiquement mettre l'application en ligne.	
		Ansible	Configure automatiquement les serveurs sans tout faire à la main.
		Prometheus / Nagios	Surveillent les serveurs et l'application pour vérifier que tout fonctionne bien. Monitoring = surveillance du système pour détecter les problèmes.

Pipeline CI/CD : CI et CD (tâches , les outils et les étapes ,objectifs)

Partie	Tâches	Outils	Étapes simples	Objectifs
CI (Intégration Continue)	- Vérifier que le code fonctionne - Tester automatiquement - Vérifier la qualité du code	- Git / GitHub / GitLab : envoyer le code - Jest / JUnit : tester le code - SonarQube : vérifier la qualité	1. Développer une fonctionnalité 2. Envoyer le code sur GitHub/GitLab 3. Les tests automatiques s'exécutent 4. SonarQube analyse le code	- S'assurer que le code marche - Détecter les erreurs rapidement - Maintenir un code propre
CD (Déploiement Continu)	- Mettre l'application en ligne automatiquement - Mettre à jour facilement	- Docker : mettre l'application dans une boîte - Kubernetes : gérer les	1. Prendre le code testé par CI 2. Créer un conteneur Docker 3. Kubernetes démarre et	- Publier les nouvelles fonctionnalités rapidement - Éviter les erreurs lors du

	- Assurer que tout fonctionne pour les utilisateurs	boîtes et copies - Jenkins / GitLab CI / GitHub Actions : automatiser le déploiement	organise les conteneurs 4. L'application est disponible pour tous les utilisateurs	déploiement - Garder l'application toujours disponible
--	---	--	---	--

Version minimaliste du fichier gitlabci.yml

stages:

- build
- test
- deploy

build_job:

stage: build

script:

- echo "Construction de l'application"

artifacts:

paths:

- dist/

cache:

paths:

- node_modules/

test_job:

stage: test

script:

- echo "Exécution des tests"

only:

- main

deploy_job:

stage: deploy

script:

- echo "Déploiement de l'application"
only:
- main

Tableau des mots-clés et leur rôle

Mot-clé	Rôle expliqué simplement
stages	Définit les étapes du pipeline (build → test → deploy).
job (ex: build_job)	Nom de la tâche à exécuter.
stage	Indique à quelle étape le job appartient.
script	Liste les commandes à exécuter dans le job.
artifacts	Fichiers générés par le job qu'on veut garder pour les autres jobs (ex: dist/).
cache	Fichiers que GitLab garde entre les jobs pour aller plus vite (ex: node_modules/).
only	Indique quand le job doit s'exécuter (ex: seulement sur la branche main).

Autrement :

Jobs et stages

- **stages** → grandes étapes : **build** → **test** → **deploy**
- **job** → une tâche dans le stage (ex: build_job)
- **stage** → dit à quel stage appartient le job
- **script** → commandes que le job doit exécuter (ex: `npm install`)
- **only** → quand le job doit tourner (ex: seulement sur la branche main)

Artifacts vs Cache

Artifacts

- Ce sont des fichiers **qu'on veut garder pour les autres jobs ou pour télécharger**.
- Exemple : ton dossier `dist/` après le build.
- Durée de vie : juste pendant le pipeline.

Cache

- Ce sont des fichiers **qu'on garde pour aller plus vite**, pour ne pas tout recalculer ou réinstaller.
- Exemple : `node_modules/` pour ne pas refaire `npm install`.
- Durée de vie : plusieurs pipelines, pour réutilisation.

Truc simple pour retenir :

- **Artifacts** = fichiers à partager
- **Cache** = fichiers pour gagner du temps

Phase du pipeline CI/CD où on utilise SonarQube

- **SonarQube** est utilisé **dans la phase CI (Intégration Continue)**.
- **Quand exactement ?** Après que le code a été écrit et envoyé sur Git (GitHub/GitLab), **avant ou pendant les tests**, pour analyser automatiquement la qualité et la sécurité du code.

En pipeline :

Build → SonarQube (analyse) → Tests → Déploiement

Objectifs de **SonarQube**

Objectif	Explication simple
Vérifier la qualité du code	Détecte les bugs, le code mal écrit ou compliqué.
Trouver les failles de sécurité	Alerte si ton code peut être vulnérable aux attaques.
Réduire la dette technique	Aide les développeurs à améliorer le code petit à petit.
Suivi dans le temps	Permet de voir si la qualité du code s'améliore ou se détériore.

Étapes simples d'installation de SonarQube

1. **Télécharger SonarQube**
 - Sur le site officiel de SonarQube, prends la version Community (gratuite).
2. **Installer Java**
 - SonarQube a besoin de Java pour fonctionner.
3. **Lancer SonarQube**
 - Exécuter le serveur SonarQube (`startSonar.bat` sur Windows ou `sonar.sh` sur Linux/Mac).
 - Il tourne sur un navigateur, par défaut `http://localhost:9000`.
4. **Créer un projet dans SonarQube**
 - Donner un nom à ton projet, récupérer le **token d'analyse**.
5. **Configurer ton pipeline CI**
 - Ajouter SonarQube à ton fichier CI/CD (`gitlab-ci.yml` ou Jenkinsfile) pour analyser le code automatiquement après chaque commit.

Version ultra simple à retenir :

- **Phase CI** → SonarQube analyse ton code.
- **Objectif** → détecter bugs, failles et code compliqué.
- **Installation** → Télécharger → Installer Java → Lancer serveur → Créer projet → Intégrer au pipeline.

Exemple minimaliste GitLab CI avec SonarQube

```
stages:
  - build
  - test
  - analyze
  - deploy

build_job:
  stage: build
  script:
    - echo "Construction de l'application"

test_job:
  stage: test
  script:
    - echo "Exécution des tests"

sonarqube_job:
  stage: analyze
  image: sonarsource/sonar-scanner-cli:latest
  script:
    - sonar-scanner -Dsonar.projectKey=my_project -
Dsonar.sources=. -Dsonar.host.url=http://sonarqube:9000 -
Dsonar.login=$SONAR_TOKEN
  only:
    - main

deploy_job:
  stage: deploy
  script:
    - echo "Déploiement de l'application"
  only:
    - main
```